Tema 2

Representación de la Información

Contenido

1. Introducción
2. Clasificación
3. Datos numéricos
    1. Naturales
    2. Corrección de errores
    3. Enteros
    4. Flotantes
4. Datos Alfanuméricos
5. Instrucciones y direcciones
    1. Modos de direccionamiento
    2. Formato de instrucciones
    3. Juego de instrucciones

2. 1 Representación de la Información

- ¿Qué es información?
  - Instrucciones: Lo que hay que hacer
  - Datos (sobre los que actúan las instrucciones)
- Representación: Traducción a un lenguaje específico
  - ...que la máquina pueda "entender" y "manipular".

(Se utilizará este símbolo para indicar la operación de representar)

- ¿Cómo se representan instrucciones y datos?
  - Utiliza un "lenguaje" con sólo dos "letras": 0 , 1
  - (cada "letra" está asociada a un potencial eléctrico)
  - Las palabras son cadenas binarias: 00001010101

¿Qué tipos de instrucciones?

- Muy simples. Ejemplo:
  - Almacenar ciertos *datos* en registros
  - Almacenar ciertos *datos* en alguna posición de memoria
  - Realizar una operación elemental con los *datos* y guardar el resultado en algún sitio (memoria o registro)
    - sumas, restas, and, or
    - opuesto, inverso, not, etc...
- Se representan en binario: NOT Reg3 → 01001 00011

¿Qué tipo de datos?

- Letras del alfabeto y otros símbolos
  - "alfanuméricos": a, B, ñ,), ?, +, 3, $, ~, ♥, ↵
  - Cadenas de alfanuméricos: "pepe", "xDD"
- Números
  - Naturales: 1, 25
  - Enteros: -12, +48
  - Racionales y reales

    Algunos no son representables: π

00001101

00110000

01010000
01000101
01010000
01000101

11.001001010100010101010010010101010101010100100.....

Dos tipos de problemas

- Problemas directos: ¿Cómo representamos X?

X → 00100100

- Problemas inversos ¿Qué representa 0010100?

00100100 → X

Inconvenientes:

- Símbolos iguales para instrucciones y datos
- 0010100101 ?
- El "contexto" es siempre necesario:
  - donde debe haber instrucciones, habrá instrucciones
  - donde se espera un dato de cierto tipo, allí debe estar
  - en caso contrario:

    "El programa ha efectuado una operación no valida"

Ejemplo de problema inverso

- ¿Qué representa 01000101?
  - No hay respuesta valida

- ¿Qué representa 01000101...

  ... si identifica un símbolo alfabético utilizando el formato de representación "ASCII"

  01000101 :"E"

2.2 Clasificación de la información

- Datos
  - Numéricos
    - Naturales (sin signo)
      - No posicionales
      - Posicionales
    - Enteros (con signo)
      - Signo-Magnitud
      - Negativos en complemento a 1, a 2
      - En exceso
    - Punto flotante (subconjunto de los reales) IEEE-754
  - Alfanuméricos
      - ASCII
      - Unicode
- Instrucciones

2.3 Representación de datos numéricos

Números Naturales

- Arbitrarios:
  - Ejemplo:

    1 → 010001

    2 → 001010

    3 → 100101
- No arbitrarios

  Responden a una ciertas reglas

  1 → 000001

  2 → 000010

  3 → 000100

Naturales

- Posicionales
  - El valor de un dígito (0 o 1) depende de la posición:

	a b
0	00
1	01
2	10
3	11

(el dígito a tiene peso 2, el dígito b, peso 1)

- Similar a la numeración *indoarábiga*: binario natural

Cuenta en binario

- 0
- 1
- 10
- 11
- 100
- 101
- 110
- 111
- 1000

Cuenta en binario

- 0000
- 0001
- 0010
- 0011
- 0100
- 0101
- 0110
- 0111
- 1000

Ojo!!!
La longitud de la secuencia es muy importante!

Binario Natural

- El peso de cada dígito depende de su valor d (0, 1) y su posición *i*

dígito	0 1 0 1 0 1
posición	*5 4 3 2 1 0*

- El peso de cada dígito es $d_i \cdot 2^i$

- El número es $0 \cdot 32 + 1 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 21$

Binario natural. Solución de problemas

- Imprescindible especificar el número de dígitos en el problema directo
- Completar con ceros a la izquierda
- Para el problema inverso: $\Sigma\ (d_i \cdot 2^i)$
- Problema directo:

23 : 2 = 11, resto 1
11 : 2 = 5, resto 1
5 : 2 = 2, resto 1
2 : 2 = 1, resto 0
1 : 2 = 0, resto 1
0 : 2 = 0, resto 0

se repite cuanto sea necesario

solución: 00010111

Binario Natural. Trucos imprescindibles

- Los ceros a la izquierda no cuentan: 001100 equivale a 1100
- Cualquier número en la forma 100000 es una potencia de 2
  - en base 10 sería una potencia de 10 (evidente, ¿no?)
- Cualquier número en la forma 111111 es una potencia de 2, menos 1
  - en base 10, ¿existe una norma equivalente?
- Del 0 al 15 deben memorizarse (0000 a 1111)
- 1K= 1024 = 2^{10} =10000000000 (1 seguido de 10 ceros)
- 1Mega= 1048576 = 2^{20} =100000000000000000000 (20 ceros)
- 1Giga= 1024 = 2^{30} =10000000000000000....000000 (30 ceros)
- Todos los impares terminan en 1, y en cero los pares
- Si A=00010101 termina en 01, entonces A módulo 4 es 1 (01)
- Si A=00010101 termina en 101, entonces A módulo 8 es 5 (101)

Problema

- En un torneo de tenis, por el sistema de eliminatorias, participan 1024 jugadores. ¿Cuántos partidos hay que programar, incluida la final?

Solución:

- 512 partidos en primera ronda + 256 partidos en segunda ronda + 128 en tercera ronda + 64 partidos en la cuarta + 32 partidos + 16 + 8 de partidos de octavos + 4 cuartos de final + 2 semifinales + final

Otra solución

- Si en cada partido se elimina un jugador, y tras la final, hay 1023 eliminados y un solo ganador:

  1023 partidos jugados

Binarios naturales. Regla

$2^n - 1 = 2^{n-1} + 2^{n-2} + 2^{n-3} + \ldots + 4 + 2 + 1$

Ejemplo: $1+2+4+8 = 15 = (16 - 1)$

$1111 = 10000 - 1$

Binario natural. Otras bases potencias de 2

- En otras bases, la cuenta es similar
  - base cuatro: 0, 1, 2, 3, 10, 11, 12, 13, 20
  - base ocho 0, 1, 2, 3, 4, 5, 6, 7, 10, 11

- Agrupaciones de bits:
  - 00 00 11 10 en binario natural es 0 0 3 2 en base cuatro

  Se agrupan los bits de dos en dos y se convierten los grupos
  - 00 001 110 es 0 1 6 en base ocho

Hexadecimal, o base 16

- Se necesitan 16 símbolos
- 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
- 0110 1000 en binario natural es 68 en base 16
- $1\cdot 2^6+1\cdot 2^5+1\cdot 2^3 = (0\cdot 2^3+1\cdot 2^2+1\cdot 2^1+0\cdot 2^0)\cdot 2^4$

  $+ (1\cdot 2^3+0\cdot 2^2+0\cdot 2^1+0\cdot 2^0)\cdot 2^0$

  $= 6\cdot 16^1+8\cdot 16^0$

- Se puede deducir una regla general de agrupación de bits
- E incluso extrapolar a base 100:

♋ 4 ♎ ♍ = 69042299

Cuatro operaciones para recordar

- Exceso a E de N → N+E
  - *El exceso a 3 de 8 es 11*
- Complemento a C de N → C-N
  - *El complemento a 1000 de 327 es 673*
- Complemento a 1 de N (binario) →
    - Cambiar todos los ceros por unos
    - Cambiar todos los unos por ceros
  - *El complemento a 1 de 010001 es 101110*
- Paridad de N (binario) →
    - Si el número de unos es par, 0
    - Si el número de unos es impar, 1
  - *La paridad de 0111001 es 0 (par)*

Otras representaciones de N

- Exceso a X (caso particular X=3): N+X en *binario*
- Complemento a X (caso, $X=2^n-1$): X-N en *binario*
- Binario natural con bit de paridad: 1 bit adicional, xor
- Códigos Hamming (SEC y DEC): más bits.
- BCD: cada digito decimal es convertido a *bn de 4 bits*
- Gray, reflejo: 000 001 011 010 110 111 101 100

- Problema: ¿Cuáles son posicionales y cuáles no?

Binario natural con bit de paridad

- Paridad de una cadena binaria:

  Paridad del número de “unos” contenidos en la cadena.
- Ejemplo: A=100011101 tiene 5 “unos”; 5 es impar, luego A tiene paridad impar

- Bit de paridad: 0 para cadenas pares, 1 para las impares (el bit de paridad de A es 1)
- Ejemplo: Representación de 23 (10111 en *bn*) con bit de paridad a la derecha. Solución: 10111 0

Representación de datos numéricos enteros (números con signo)

Representaciones elementales (1)

- Signo – Magnitud
  - 1 bit representa el signo (siempre se usa el de la izquierda)
    - 0 es positivo
    - 1 es negativo
  - n-1 bits representan la magnitud (en binario natural)
  - Doble representación para el cero. Ejemplo de 8 bits:
    - 00000000
    - 10000000

Notación:

n → es el número de bits, incluyendo siempre el signo

N→ es el número que se quiere representar

Representaciones (n=4)

base 10	signo magnitud				
+7	0 111				
+6	0 110				
+5	0 101				
+4	0 100				
+3	0 011				
+2	0 010				
+1	0 001				
+0	0 000				
-0	1 000				
-1	1 001				
-2	1 010				
-3	1 011				
-4	1 100				
-5	1 101				
-6	1 110				
-7	1 111				
-8					

Representaciones elementales (2)

- Representaciones en exceso
  - Una representación en exceso a X de n bits permite representar números desde –X hasta 2^n-X
- Ejemplo: exceso a 127 con 8 bits

-127 → 00000000 (-127 +127 en *bn*)

0 → 01111111 (0 +127 en *bn*)

128 → 11111111 (128 +127 en *bn*)

- El bit más significativo indica el signo, pero 1 es +
- El cero tiene representación única

Representaciones

base 10	signo magnitud	exceso a 8	exceso a 7		
+7	0 111	1111	1110 ↑		
+6	0 110	1110	1 101		
+5	0 101	1 101	1100		
+4	0 100	1100	1 011		
+3	0 011	1 011	1 010		
+2	0 010	1 010	1 001		
+1	0 001	1 001	1 000		
+0	0 000	1 000	0 111		
-0	1 000				
-1	1 001	0 111	0 110		
-2	1 010	0 110	0 101		
-3	1 011	0 101	0 100		
-4	1 100	0 100	0 011		
-5	1 101	0 011	0 010		
-6	1 110	0 010	0 001		
-7	1 111	0 001	0 000		
-8		0 000			

Representación *de negativos en complemento a 2*

- Es la representación estándar para números enteros
- El bit de signo es el más significativo:
  - 01010101 → Este número de n=8 bits es positivo
  - 1001 → Este número de n=4 bits es negativo
- Si el número es positivo, se representa en binario natural
  - De 0 a 2^n-1
- Si el número es negativo…
  - La representación no es muy poco intuitiva
  - Los negativos se representan en complemento a 2^n del valor absoluto:
    - *BinarioNatural*(2^n – *abs*(N))

- La denominación auténtica es *Negativos en complemento a 2^n*

Representaciones

base 10	signo magnitud	exceso a 8	exceso a 7		negativos C–2
+7	0 111	1111	1110 ↑		0 111
+6	0 110	1110	1 101		0 110
+5	0 101	1 101	1100		0 101
+4	0 100	1100	1 011		0 100
+3	0 011	1 011	1 010		0 011
+2	0 010	1 010	1 001		0 010
+1	0 001	1 001	1 000		0 001
+0	0 000	1 000	0 111		0 000
-0	1 000				
-1	1 001	0 111	0 110		1 111
-2	1 010	0 110	0 101	Complemento a 16 de 2 →	1 110
-3	1 011	0 101	0 100		1101
-4	1 100	0 100	0 011		1 100
-5	1 101	0 011	0 010		1 011
-6	1 110	0 010	0 001	Complemento a 16 de 6 →	1 010
-7	1 111	0 001	0 000		1 001
-8		0 000		Complemento a 16 de 8 →	1 000

Negativos en complemento a 2

- ¿Porqué un formato tan poco intuitivo?

- **Motivo nº 1: aritmética mucho más sencilla**.

Veamos ejemplos equivalentes en base 10, con n=2, y negativos en complemento a 100 (y utilizo colores en lugar de signos)

Tengo 23 euros, y como es positivo lo represento — 23 euros
Gasto 12 euros en mascarillas.
Lo represento en complemento a 100: — 88 euros

¿Cuánto suman las dos cantidades?

$$\begin{array}{r} 23 \\ +88 \\ \hline 111 \end{array}$$ (n es 2)

Negativos en complemento a 2

- ¿Porqué un formato tan poco intuitivo?

- **Motivo nº 1: aritmética mucho más sencilla**.

Veamos ejemplos equivalentes en base 10, con n=2, y negativos en complemento a 100 (y utilizo colores en lugar de signos)

Debo 44 euros, y como es negativo lo represento — 56 euros

.

Me tocan 16 euros en la primitiva — 16 euros

¿Cuánto suman las dos cantidades?

56
+16
72 (Mi deuda es 28)

Negativos en complemento a 2

- ¿Porqué un formato tan poco intuitivo?

- **Motivo nº 1: aritmética mucho más sencilla**.

Veamos ejemplos equivalentes en base 10, con n=2, y negativos en complemento a 100 (y utilizo colores en lugar de signos)

Debo 44 euros, y como es negativo lo represento 56 euros

.

Ahora me llega otra factura de 23 euros 77 euros

¿Cuánto suman las dos cantidades?

$$\begin{array}{r} 56 \\ +77 \\ \hline 133 \end{array}$$

(Mi deuda es 67)

Negativos en complemento a 2

- ¿Porqué un formato tan poco intuitivo?

- **Motivo nº 2: ¡Es muy fácil calcular el complemento!**.

Veamos ejemplos equivalentes en base 10, con n=2, y negativos en complemento a 100

Pregunta: ¿Cuál es el complemento a 10000 del número **1327**?

Respuesta rápida: Uno más el complemento a 9999 del número **1327**

Vaya, pues me he quedado igual

No te creas: El complemento a 9999 es el complemento a 9 de cada dígito separado:

9 – 1 = 8
9 – 3 = 6
9 – 2 = 7
9 – 7 = 2

Respuesta final: 8673

Representaciones

base 10	signo magnitud	exceso a 8	exceso a 7		negativos C–2
+7	0 111	1111	1110 ↑		0 111
+6	0 110	1110	1 101		0 110
+5	0 101	1 101	1100		0 101
+4	0 100	1100	1 011		0 100
+3	0 011	1 011	1 010		0 011
+2	0 010	1 010	1 001		0 010
+1	0 001	1 001	1 000		0 001
+0	0 000	1 000	0 111		0 000
-0	1 000				
-1	1 001	0 111	0 110		1 111
-2	1 010	0 110	0 101	Complemento a 16 de 2 →	1 110
-3	1 011	0 101	0 100		1101
-4	1 100	0 100	0 011		1 100
-5	1 101	0 011	0 010		1 011
-6	1 110	0 010	0 001	Complemento a 16 de 6 →	1 010
-7	1 111	0 001	0 000		1 001
-8		0 000		Complemento a 16 de 8 →	1 000

Antes de seguir…

- ¿Sabemos sumar uno, a un número binario?
- ¿Qué es el acarreo/carry?
- ¿Y el desbordamiento/overflow?

Representaciones

base 10	signo magnitud	exceso a 8	exceso a 7	negativos C –1	
+7	0 111	1111	1110 ↑	0 111	
+6	0 110	1110	1 101	0 110	
+5	0 101	1 101	1100	0 101	
+4	0 100	1100	1 011	0 100	
+3	0 011	1 011	1 010	0 011	
+2	0 010	1 010	1 001	0 010	
+1	0 001	1 001	1 000	0 001	
+0	0 000	1 000	0 111	0 000	
-0	1 000			1 111	
-1	1 001	0 111	0 110	1 110	
-2	1 010	0 110	0 101	1101	
-3	1 011	0 101	0 100	1 100	
-4	1 100	0 100	0 011	1 011	
-5	1 101	0 011	0 010	1 010	
-6	1 110	0 010	0 001	1 001	
-7	1 111	0 001	0 000	1 000	
-8		0 000			

Representación “negativos en complemento a 2”

- Negativos: complemento a 1, **más uno**
- Ejemplo: -23 con 6 bits

  (recordemos *bn(23)=10111*)

  Signo: 1

  C-1(23)= 01000 → C-2(23)= 01000+1= 01001

  Resultado: 1 01001

¡ Aviso Importante !

¡Sólo los negativos invierten los bits!

Representación “negativos en complemento a 2”

- Problema: ¿Qué representa 010111, si está en formato N-2?
- Respuesta: +23 (¡no invertimos bits, porque es positivo!)

- Problema: ¿Qué representa 100111, si está en formato N-2?

Representación “negativos en complemento a 2”

- ¿Qué representa 100111, si está en formato N-2?
  - Identificar signo: *es negativo*
  - Si es negativo: Por definición 00111 (7) es el complemento a 32 de 25; la solución es -25
  - Alternativa más fácil: si es negativo, al resto de la cadena:

a) Restar 1	00110	
b) Invertir bits	11001	
c) Convertir de *bn* a base-10	25.	La solución es -25

  - Otra más fácil aún: si es negativo , al resto de la cadena :

a) Invertir bits	11000	
b) Sumar 1	11001	
c) Convertir de *bn* a base-10	25.	La solución es -25

Extensión de números y extensión del signo

- Dada una **representación** con n bits de un cierto número ¿cómo se representaría con n+m bits?

- s-m: Insertar m ceros entre el signo y la magnitud
- exceso: Insertar m ceros a la izquierda
- c-1 y c-2: Repetir el signo m veces a la izquierda

"Extensión del signo"

Números en punto fijo

- Naturales y enteros se conocen como

  "Números en punto fijo, o coma fija"
- La coma de fracción está **fija** en el extremo derecho de los registros de almacenamiento:

1	1	0	1	1	1

,

- Se puede, no obstante, **fijar** en otra posición:

Números en punto fijo

1	1	0	1	1	1

,

- Posiciones de los dígitos: *i= -2..3*
- Pesos a la derecha de la coma: $d_i \cdot 2^i$ (igual)
- En el ejemplo, los pesos son 0.5 y 0.25
- Para ejercicios sobre complemento a 2, se suma 1 al dígito más a la derecha.
- El problema directo es diferente para las partes entera y fracción:

Convertir 2.5625 a *bn de 4,4 bits*

- Parte entera:

2:2=1 resto 0

1:2=0 resto 1

0:2=0 resto 0

0:2=0 resto 0

- Parte fracción:

0.5625*2= 1 . 125

0.125 *2= 0 . 25

0.25 *2= 0 . 5

0.5 *2= 1 . 0

0	0	1	0 ,	1	0	0	1

Extensión de números por la derecha

- A la derecha de la coma, la extensión con m bits es:
  - s-m y n-2: añadir m "ceros"
  - n-1:
    - añadir m "ceros" si es positivo,
    - m "unos" si es negativo

Ejemplos (de 8 a 16 bits)

Sin fracción:

Consideremos el 23 y el -23

Formato	8 bits	16 bits	
Binario natural	00010111.	0000000000010111.	0's a izqda
S-M	00010111.	0000000000010111.	0's a izqda. magnitud
N-1	00010111.	0000000000010111.	Ext. de signo
N-2	00010111.	0000000000010111.	Ext. de signo

Formato	8 bits	16 bits	
Binario natural	No valido	No valido	
S-M	10010111.	1000000000010111.	0's a izqda. magnitud
N-1	11101000.	1111111111101000.	Ext. de signo
N-2	11101001.	1111111111101001.	Ext. de signo

Ejemplos (de 8 a 16 bits)

Con fracción: 6.2 → 10.6

Consideremos el 23.25 y el -23.25

Formato	8 bits	16 bits	
Binario natural	010111.01	0000010111.010000	0's a izqda., 0's a dcha
S-M	010111.01	0000010111.010000	0's a izqda. Magnitud, 0's dcha
N-1	010111.01	0000010111.010000	Ext. de signo, 0's a dcha
N-2	010111.01	0000010111.010000	Ext. de signo, 0's a dcha

Formato	8 bits	16 bits	
Binario natural	No valido	No valido	
S-M	110111.01	1000010111.010000	0's a izqda. Magnitud, 0's dcha
N-1	101000.10	1111101000.101111	Ext. de signo, **1's a dcha!!!**
N-2	101000.11	1111101000.110000	Ext. de signo, 0's a dcha

Representación de datos numéricos reales (números en punto flotante)

Coma flotante

- La representación en punto fijo no es apropiada para ciertos números:
  - $6.023 \cdot 10^{23}$, $1.6 \cdot 10^{-19}$, etc...
- Es preferible usar la forma: mantisa, base, exponente
- El exponente hace "flotar la coma" de la mantisa

Coma flotante

- Si la mantisa está representada en una base B (p.e. B=2), entonces la base en punto flotante debe ser una potencia de B:

$1.011 \cdot 2^{12} = 10.11 \cdot 2^{11}$

$1.011 \cdot 4^{6} = 101.1 \cdot 4^{5}$

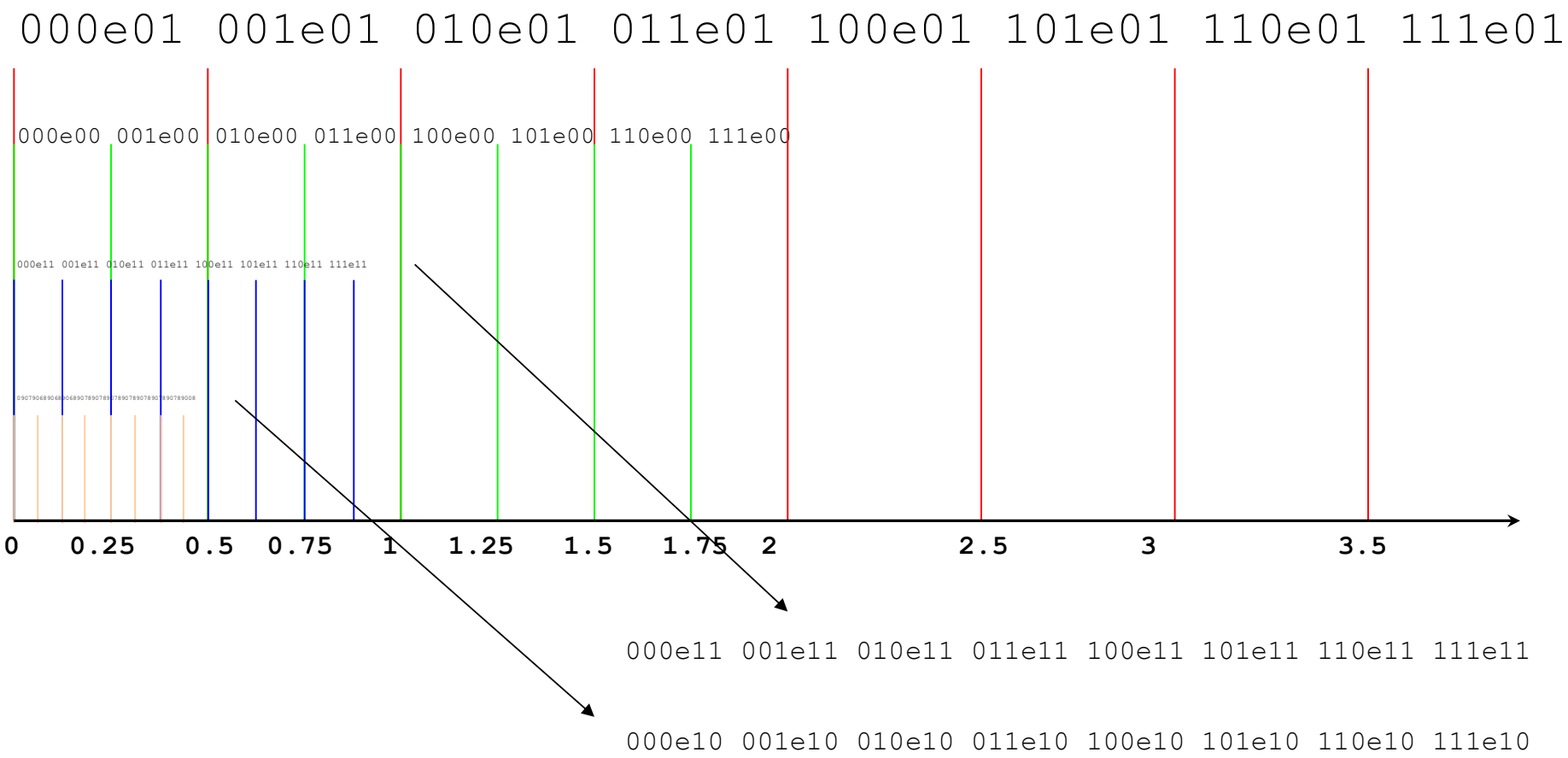
- **I**ncrementos y **D**ecrementos de exponente implican desplazamientos a **I**zquierda y **D**erecha de la coma *un número* ***entero*** *de posiciones*

Coma flotante

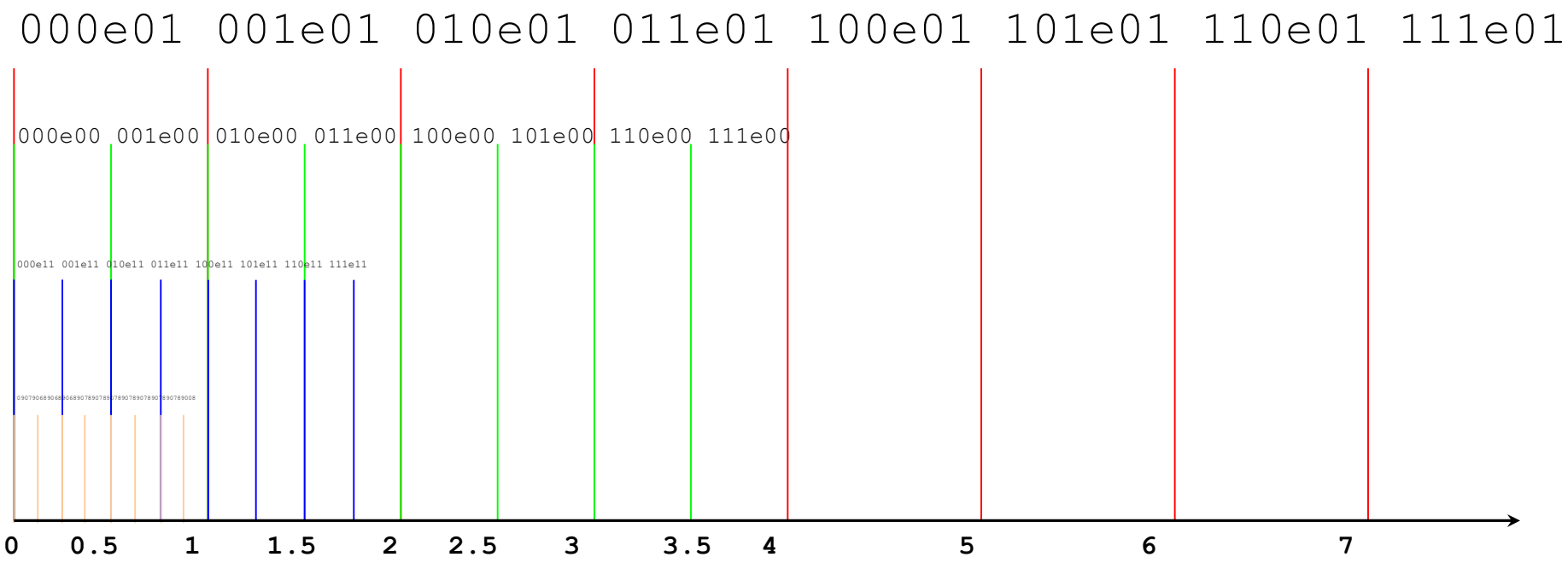
- Fijada la base, un número en coma flotante es una pareja de números en punto fijo: Mantisa-Exponente
- Ejemplo:
  - Base 2
  - 3 bits de mantisa *bn*, en la forma *m,mm*
  - 2 bits exponentes (representados en C-2)

- 1,00 e 11 ⟶ $1.0 \cdot 2^{-1} = 0.5$
- 1.10 e 01 ⟶ $1.5 \cdot 2^{1} = 3$

Coma Flotante, Nº representables



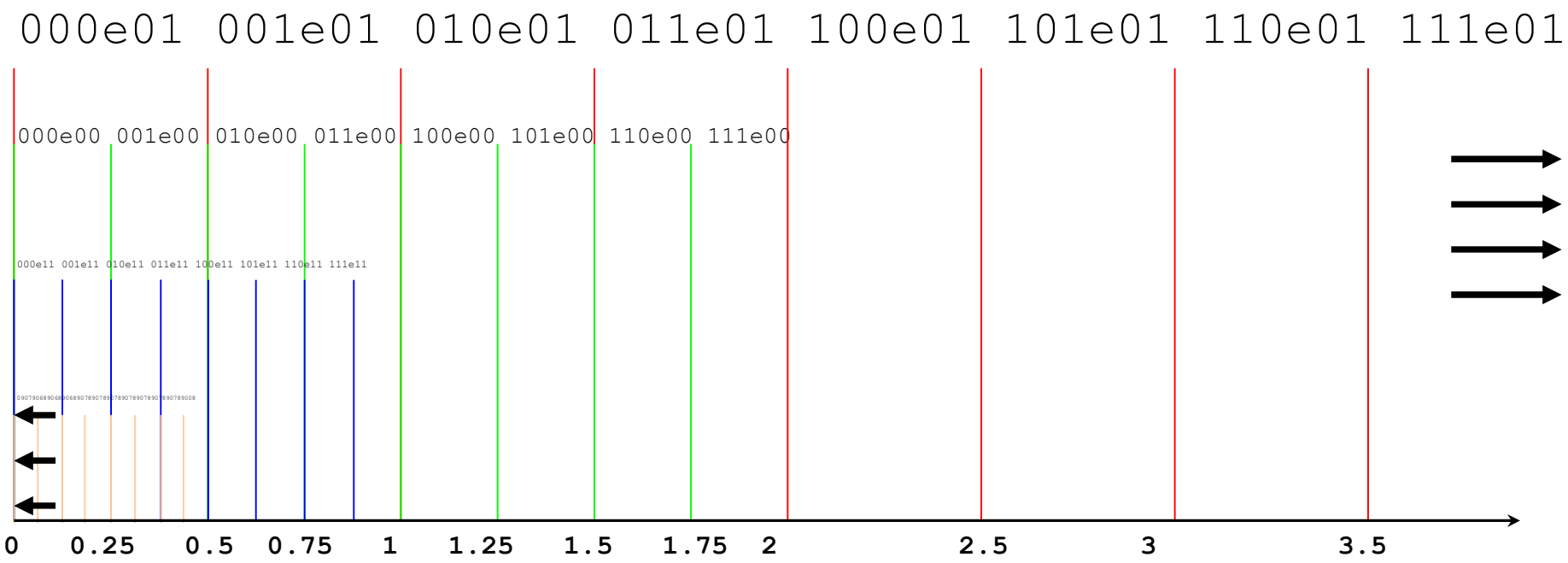
Coma Flotante. Bases superiores. Ej. 4



Mayor rango de representación. Menor precisión

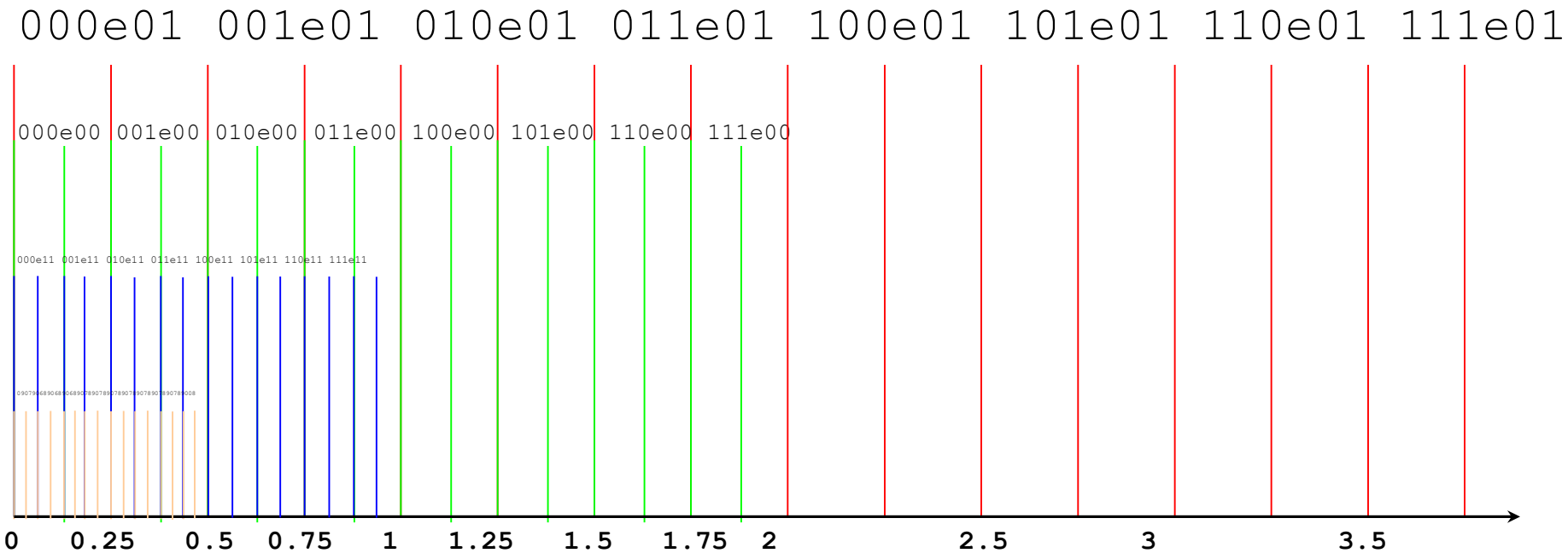
(misma cantidad de números representables, pero más dispersos)

Coma Flotante. Aumento del exponente



Mayor rango de representación. Mayor proximidad al cero.
Misma precisión

Coma Flotante. Aumento de mantisa



Mismo rango de representación. Mayor precisión

Normalización de mantisas

- 6,023e23 60,23e22 602,3e21 varias formas de representar un mismo número
- Ejemplo: 5 bits → 32 combinaciones; pero sólo 20 números representables (redundancia)
- Solución: Fijar la coma respecto al dígito más significativo (DMS): **Normalización.**
- Ej.: Sólo la forma 6.023e23 es válida
- El cero es un caso especial (no tiene DMS)

Normalización de mantisas. Bit implícito

- Sólo las combinaciones: 1MM-EE son válidas
- "Gastamos" un biestable para guardar siempre un 1
- Solución: Al representar, no guardar ese 1

  - Bit implícito: 1.011e 11 se guarda como 011e11

(aumentamos la precisión sin coste, al evitar la redundancia)

¡Al operar con el número, hay que recuperar el bit implícito!

IEEE-754 de 32 bits

- Exponente E en exceso a 127 con *8 bits*
- Mantisa en formato signo S *(1 bit)* - magnitud M
- La magnitud M tiene 24 bits, pero está normalizada con BMS a la izquierda de la coma, y el BMS se hace implícito *(23 bits)*

$$(-1)^{S} \cdot 1.M \cdot 2^{(E-127)}$$

S	EEEEEEEE	*MMMMMMMMMMMMMMMMMMMMMMM*

IEEE–754 de 32 bits

- 8 bits para exponentes (rangos de 10±38)
- 23 bits para mantisa (precisión equivalente a 7 dígitos en base 10)

```
float a=3.14159263523452345;
main()
{
printf("%22.20f\n",a);
}
3.14159274101257324219
```

Conversión de N a IEEE-754

- Paso 1. Identificar el signo S
- Paso 2. Expresar el número en formato mantisa · $2^{\text{exponente}}$
  - Ya está así! (vete al paso 3)
  - Si es entero y relativamente pequeño, exprésalo como mantisa · 2^0
  - Igual si es entero y fracción, o fracción grande
  - Lo siguiente no es frecuente, y mucho menos en exámenes de esta asignatura:
    - Si es un entero relativamente grande, divídelo sucesivamente por 2
    - Si es fracción y relativamente pequeña, multiplica sucesivamente por 2
    - Si es entero y muy grande, o fracción muy pequeña, expresado como m ·10^{exp}

La forma más rápida suele consiste en determinar un valor aproximado para x en esta fórmula

$2^x \simeq 10^{\text{exponente}}$ y a partir de ahí, calcular el coeficiente que permita la conversión

Conversión de N a IEEE-754 (2)

- Paso 3. (Innecesario) Intenta facilitar el paso 4
  - Si la parte fracción de la mantisa es 0.5, 0.25, 0.125, 0.0625 o alguna de sus sumas combinadas, lo más probable es que se pueda convertir a un entero. (Prueba a multiplicar la mantisa por 2 sucesivamente, a ver si se simplifica)
    - Ejemplo 2.875 = 23 ·2-3
- Paso 4. Convierte la mantisa a binario, usando el procedimiento ya conocido
- Paso 5. Normaliza, flotando la coma y cambiando el exponente
- Paso 6. Suma 127 al exponente y representa en bn con 8 bits!

Conversión desde IEEE754

- Simplemente, Convierte el exponente a decimal y aplica la expresión

$$(-1)^{S} \cdot 1.M \cdot 2^{(E-127)}$$

- Y si es necesario, convierte la mantisa a base 10

Casos especiales

- E=255 (11111111)
  - M=0
    - S=0 $\Rightarrow +\infty$
    - S=1 $\Rightarrow -\infty$
  - M≠0 $\Rightarrow$ NaN (Not A Number)
- E=0 (00000000)
  - M=0 $\Rightarrow$ 0
  - M≠0 $\Rightarrow (-1)^s \cdot 0.M \cdot 2^{-126}$ (no está normalizado)

IEEE–754 de 64 bits

- 11 bits para exponentes (rangos de 10±308 !)
- 52 bits para mantisa (precisión equivalente a 15 dígitos en base 10)

```
double a=3.14159263523452345;
main()
{
printf("%22.20f\n",a);
}
3.14159263523452336742
```

Representación de datos alfanuméricos

ASCII

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	SPC	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

UNICODE

- Lista ordenada de todos los caracteres conocidos
  - Universal Character Set
  - Cada símbolo tiene asignado un número de 32 bits: Code Point
- ¿8bit, 16bit, 32bit?
  - Opc. A: Compromiso entre eficiencia y universalidad.
  - Opc. B: UTF-8 ✓
    - Utilizar 1 (ascii), 2 o 4 bytes (n ascii)
    - 1 byte, a menos que sea 11XXXXXX (multibyte char)
    - 10XXXXXX (continuation chars)

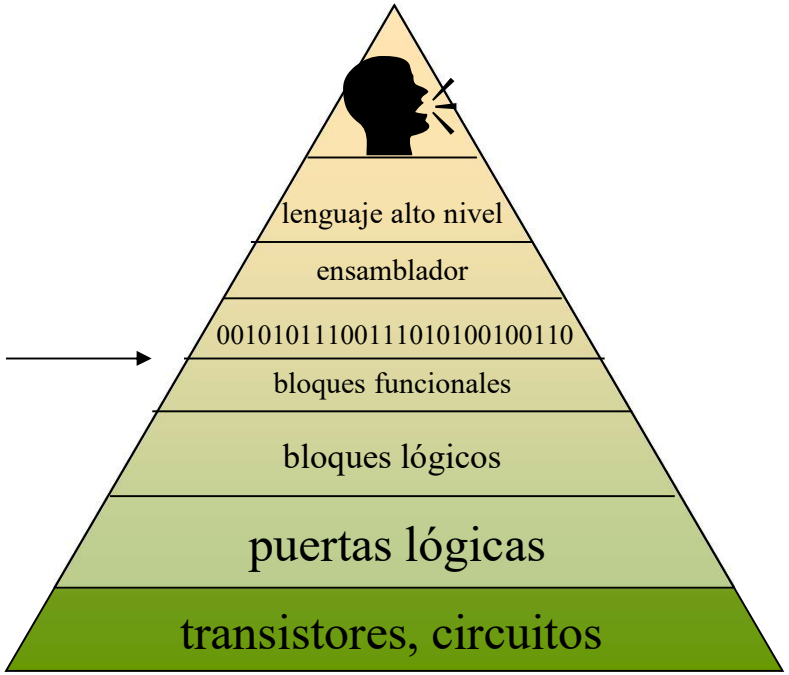
Hola este es un ejemplo:
وسايل

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00000000	48	6F	6C	61	20	65	73	74	65	20	65	73	20	75	6E	20	Hola este es un
00000010	65	6A	65	6D	70	6C	6F	3A	0D	0A	D9	88	D8	B3	D8	A7	ejemplo: Ù¦Ø³ØS
00000020	DB	8C	D9	84	20												Û¦Ù¦

- UNICODE también define UTF-16 y UTF-32
  - necesitan especificar endianismo
  - Byte Ordering Mask: FEFF (big), FFFE (little)

Instrucciones y Direcciones

Arquitectura del Juego de Instrucciones

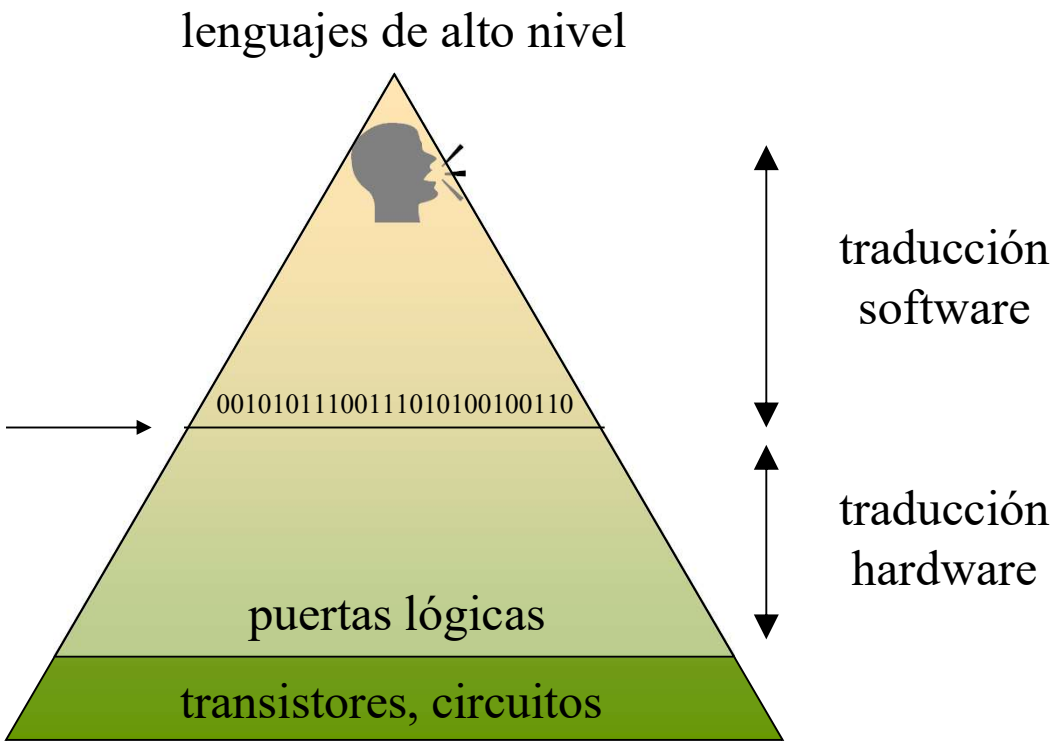


Arquitectura del Juego de Instrucciones

nivel ISA: Instruction Set Architecture:

La barrera entre el hardware y el software

Inteligible para nosotros, también para los circuitos



Las operaciones del hardware

- Operaciones que el computador realiza en el nivel que estudiamos:
  - Ejemplos:
    - Mover (en realidad, es copiar, y equivale al típico x = y de alto nivel)
    - Sumar (x = y+z)

- Instrucción: Secuencia binaria donde se indica:
    - La operación a realizar (Código de Operación)
    - Con qué datos la va a realizar
    - Ejemplo: sumar x y z: 01000 01010 01001 01001

- Lenguaje ensamblador:representación intermedia formada por:
  - Mnemónico. Ej.: ADD, MOVE, JUMP
  - Operandos - El nombre de un registro, un dato, una dirección
    ADD REG1, REG2, REG3

- La traducción es simple. Tablas específicas para diferentes *arquitecturas*
  - Ej.: ADD → 0010.

Operandos y direcciones

- Las operaciones requieren **operandos**
  - add destino, dato1 ("fuente 1"), dato2 ("fuente 2")
  - jump destino
  - mov destino, dato fuente x=y , a=17

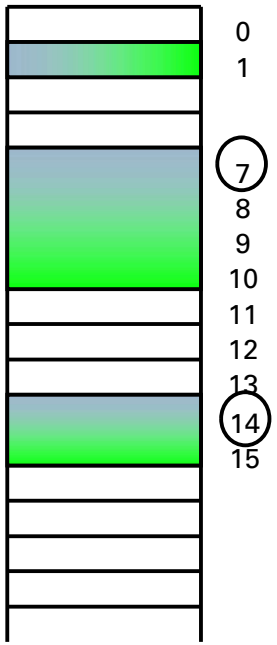
- Constantes o **variables**

  Las variables se conocen como **direcciones (residencia del dato)**
  - mov dirección_destino, dirección_fuente
  - Sólo hay dos almacenes: Registros y Memoria
- Modos de direccionamiento
  - Diferentes formas de indicar la residencia del dato
- Operandos implícitos: Ej.: INCR X

Modos de direccionamiento

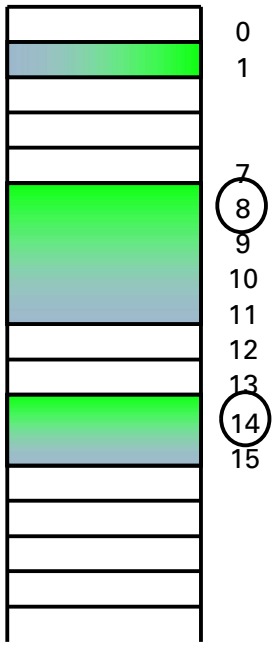
Direcciones de memoria

- La memoria es una estructura ordenada de los datos
- Un dato puede ocupar más de una posición
  - ✓ La dirección del dato es la de la posición más baja
- Los datos deben estar alineados en memoria
  - ✓ Datos de tamaño n en direcciones múltiplo de n
- Situación del dato en múltiples posiciones
  - ✓ Big "endian" ⇒ dirección (dato) = dirección (palabra más significativa)
  - ✓ Little "endian" ⇒ dirección (dato) = dirección(palabra menos significativa)



Direcciones de memoria

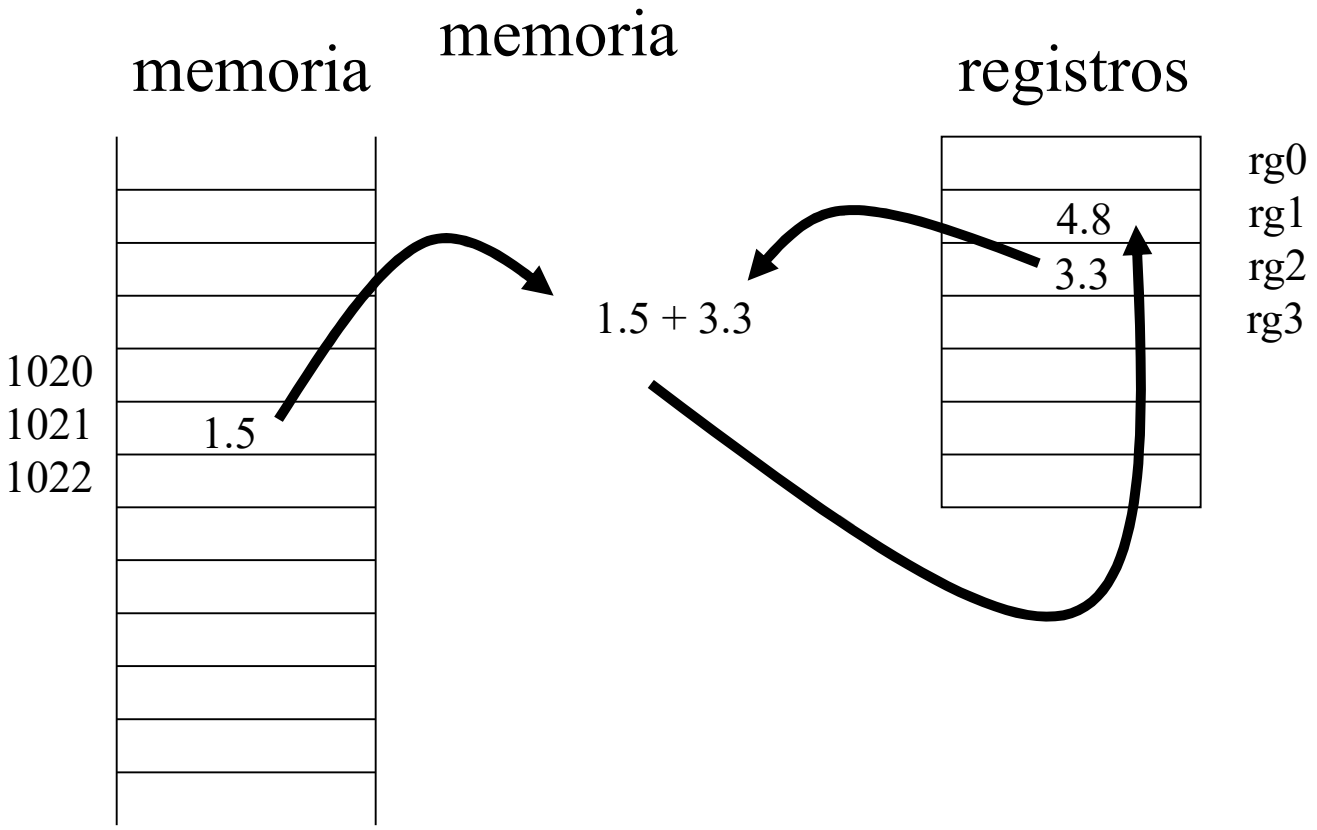
- La memoria es una estructura ordenada de los datos
- Un dato puede ocupar más de una posición
  - ✓ La dirección del dato es la de la posición más baja
- Los datos deben estar alineados en memoria
  - ✓ Datos de tamaño n en direcciones múltiplo de n
- Situación del dato en múltiples posiciones
  - ✓ Big "endian" ⇒ dirección (dato) = dirección (palabra más significativa)
  - ✓ Little "endian" ⇒ dirección (dato) = dirección(palabra menos significativa)



Modos de direccionamiento directos

- Almacén del computador: registros y memoria.
- El operando establece "directamente" la residencia del dato
  - ✓ el nombre de un registro (MD directo por registro)
  - ✓ la posición en la memoria (MD directo por memoria)
- MD directo por registro se usa en el 50% de instrucciones

ADD R1,R2,1021

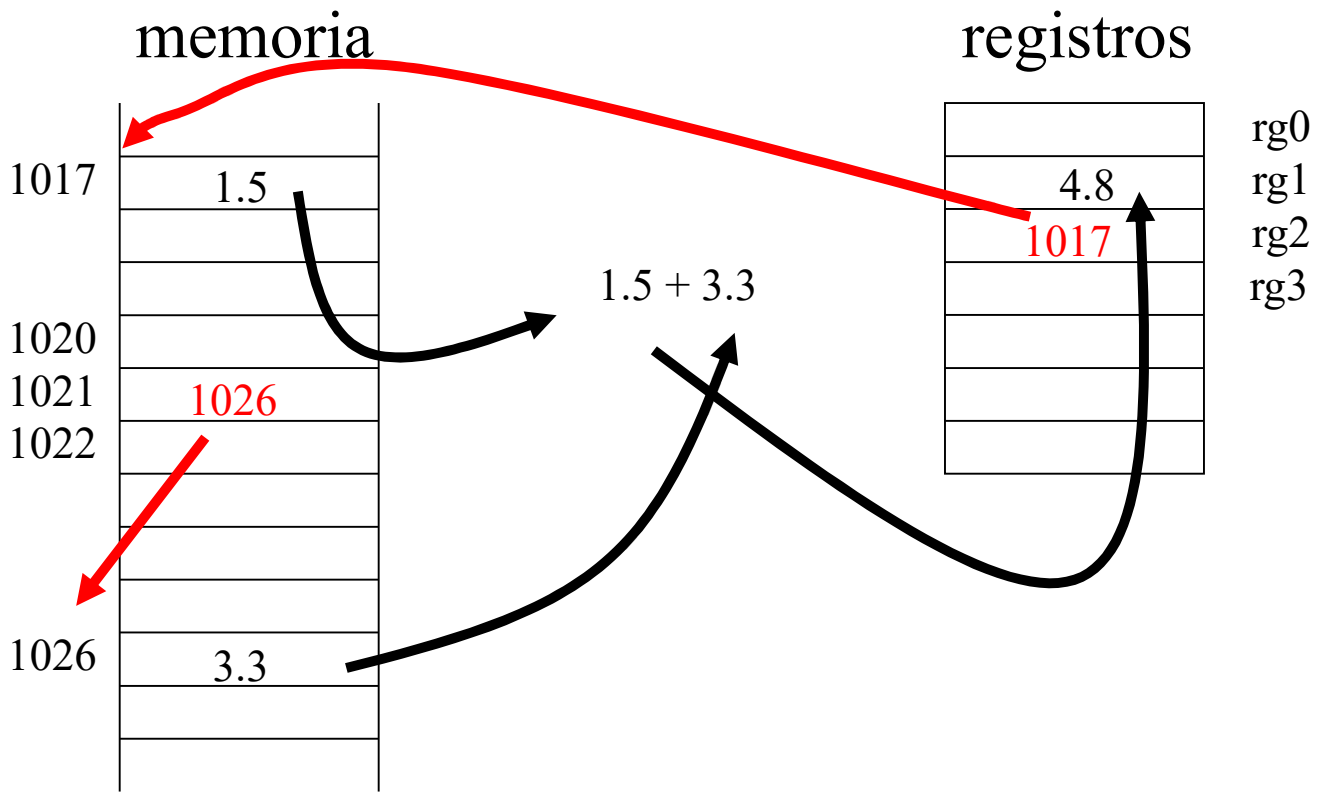


MD indirectos

- La instrucción indica el lugar donde se encuentra la dirección del dato (**"indirecciones" o punteros**)
  - ✓ un registro que guarda la dirección en memoria de un dato (**MD indirecto por registro, indirecto**)
  - ✓ una posición de memoria que guarda la dirección en memoria de un dato (**MD indirecto por memoria**)
- En los MD indirectos, el dato casi siempre se encuentra en la memoria.
- Puede haber varios grados de indirección

Modos indirectos, ejemplo

ADD R1, (R2),(1021)

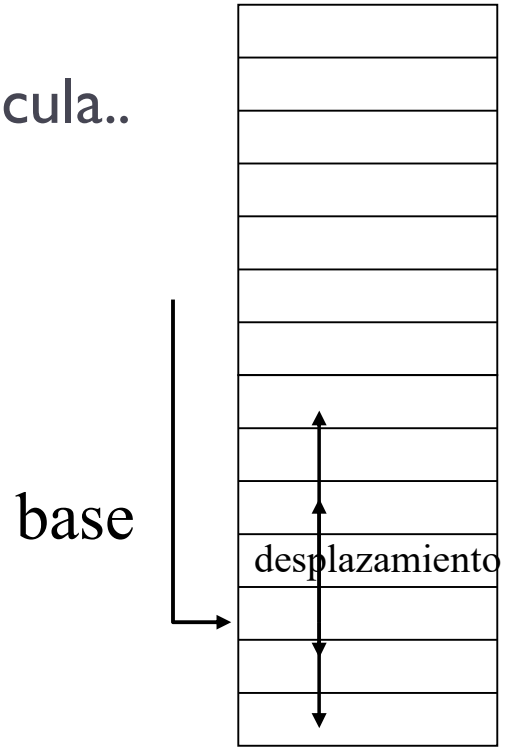


Otros modos de direccionamiento

- **MD inmediato, o literal.** Las instrucciones incluyen el propio dato

  Ej.: ADD "dirección destino", # **1.5** , # **3.3**

- **MD relativo.** La posición en memoria se calcula..
  - **Suma** de dos enteros.
    - Valor inmediato (constante entera)
    - Valor en registro (variable entera)
  - La suma debe ser un número natural
  - Si despl==0 → Modo indirecto por registro



Operandos y operaciones implícitas

Operandos implícitos:

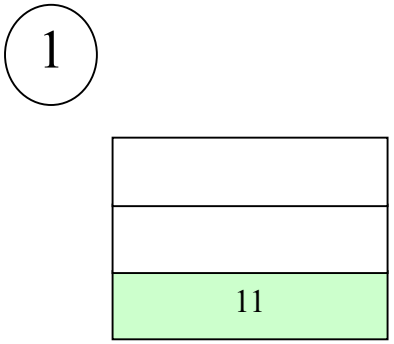
- operandos que no aparecen de forma explícita (ya sea en ensamblador o binario)
- Ejemplos:
  - INCR reg1 (reg1=reg1 + 1)
  - MUL BL (AL = AL + BL)

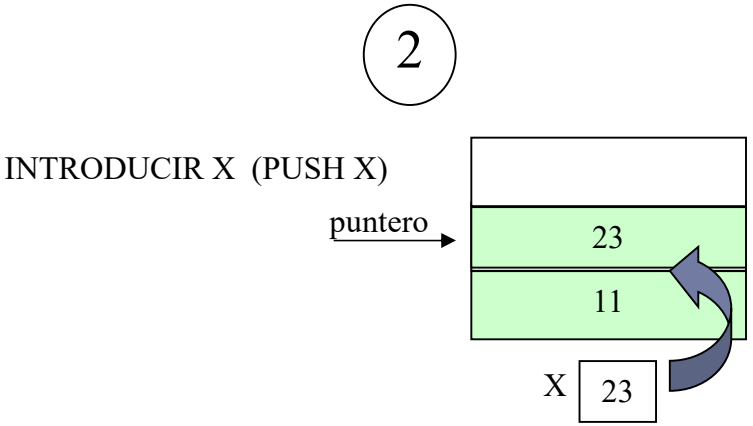
Operaciones implícitas:

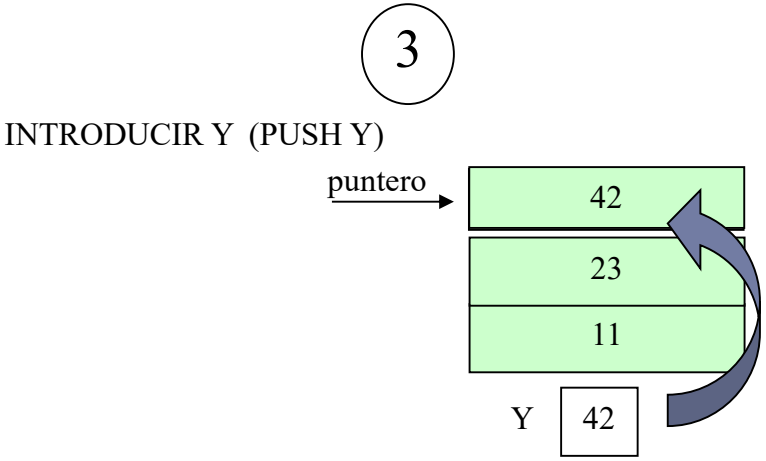
- Todas las instrucciones incrementan implícitamente el registro PC
- Muchas instrucciones modifican los registros de estado

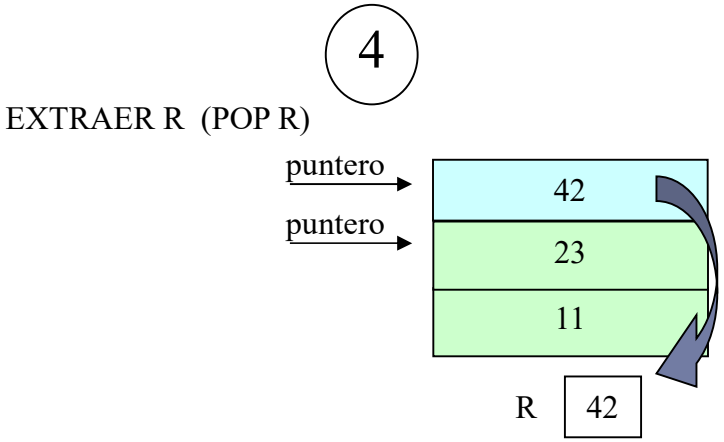
Un ejemplo que contiene operandos y operaciones implícitas: la pila

La Pila


1
11


2
INTRODUCIR X (PUSH X)
puntero
23
11
X
23


3
INTRODUCIR Y (PUSH Y)
puntero
42
23
11
Y
42


4
EXTRAER R (POP R)
puntero
puntero
42
23
11
R
42

Especificación (en ensamblador) del modo de direccionamiento

- En lenguaje ensamblador, existe una simbología especial para diferenciar los modos de direccionamiento:

  LD RG3, #23

  ADD RG5, 28(RG4)

- El uso de etiquetas para las direcciones absolutas permite la reubicación del código

  LD R2, *edad*

- Diferentes ensambladores, distintos símbolos
- Un estándar: IEEE-694

Especificación (en ensamblador) del modo de direccionamiento

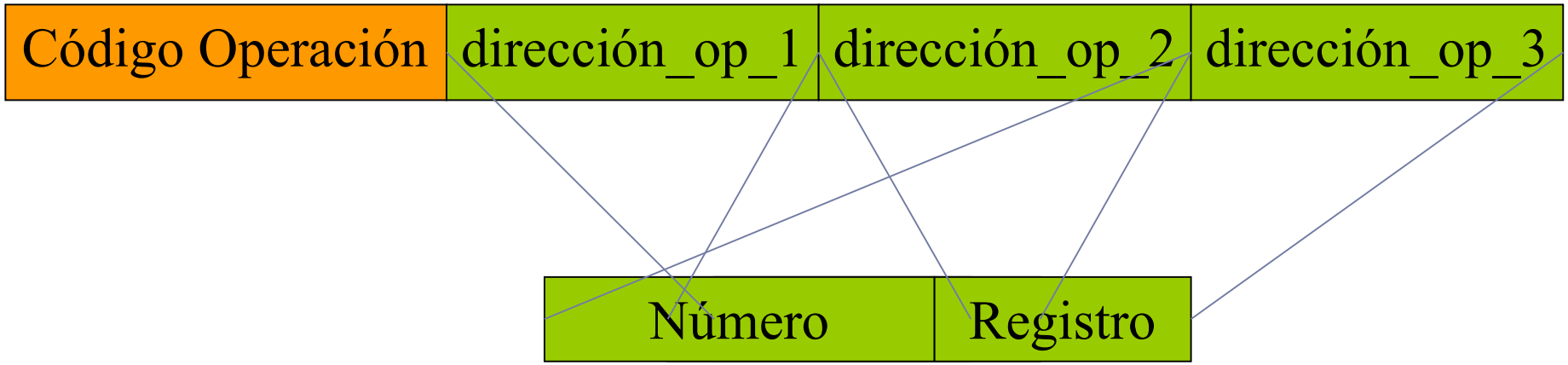
	MIPS	ejemplos	IEEE-694	ejemplo	80x86	ejemplo
Inmediato	valor	63, 0xF23	#valor	#63	valor	63
Registro	valor	$s23	.registro	.23	registro	AX
Absoluto	etiqueta	*edad*	/dirección	/1025	etiqueta	*edad*
Indirecto reg	[$registro]	[$s23]	[.registro]	[.23]	[reg]	[BX]
Relativo	desp[$reg]	100	dir[.reg]	1025[.23]	[reg]+despl	[BX]+4,*ed[*B]X]
Relativo PC			$dirección	$-8		

Formato de las Instrucciones

Juntándolo todo:

La instrucción

La instrucción es la unión de varias cadenas binarias (campos)



Longitud total de la instrucción

- Fija:
  - Múltiplo de 8 bits. Frecuentemente **32** bits.
  - 8 bits en el procesador de la primera lección
- Variable:
  - Sólo en procesadores antiguos
  - Mejor aprovechamiento:
    - Más bits para instrucciones con más operandos
  - Hardware más complejo e ineficiente (descartado)

El campo código de operación

- Indica la operación a realizar en el ciclo de instrucción actual
- Longitud de la secuencia binaria
  - log2(nº de instrucciones) → Si es un campo de longitud fija
  - Unos **6 bits** en procesadores modernos (64 instrucciones)
  - Más de 8 bits en procesadores CISC (+256)
- Codificación arbitraria. Ejemplos:
  - ADD : 010000
  - NOP: 000 (instrucción que no hace nada; frecuentemente, 000..0)
  - En caso de longitud variable, se reservan opcodes cortos a instrucciones con más argumentos (operandos), para disponer de más bits
- A veces, extensible con subcódigos de operación
  - La macroinstrucción 000000 en MIPS se extiende con 6 bits adicionales

Ejemplos

- Un computador con 4 instrucciones (2 bits)
  - 00 Sumar
  - 01 Mover
  - 10 Saltar
  - 11 Dormir

- Computador de 8 instrucciones, una de ellas con 4 funciones
  - 000 Calcular
    - 000 00 sumar
    - 000 01 restar
    - 000 10 multiplicar
    - 000 11 dividir
  - 001 Mover
  - 010 Saltar
  - 011 Dormir
  - 100 Bifurcar
  - 101 Push
  - 110 Pop
  - 111 No hacer nada (Nop)

Total: $2^3 - 1 + 2^2 = 11$ instrucciones

Problema de CASIUM

Si un computador tiene 27 instrucciones, y una de ellas usa 6 bits como campo de función. ¿cuántas instrucciones podría llegar a implementar, en realidad?

Si tiene 27 instrucciones, necesita 5 bits de código de operación, ceil(log2(27)))=5

Si tiene 5 bits de código de operación puede llegar a implementar 32 instrucciones

Pero una de ellas se reconvierte en 2^6 (64 funciones)

Suman 31+64

Los campos de operandos

- Tantos como requiera la instrucción
- Normalmente de 0 a 3 campos
  - Ejemplo 0: RET
  - Ejemplo 1: PUSH op1
  - Ejemplo 2: MOVE op1 op2
  - Ejemplo 3: ADD op1 op2 op3
- Cada campo podría precisar de subcampos:
  - Bits para nombrar un registro:
    - Si es modo directo por registro
    - Si es modo indirecto por registro
    - Si es modo relativo
  - Bits para indicar un número
    - Si es modo relativo (un desplazamiento: un entero)
    - Si es inmediato (normalmente un número entero)
    - Si es directo por memoria (un número natural). Descartado.

El modo de direccionamiento

- En los procesadores modernos, el código de operación establece el modo de direccionamiento de cada operando:
  - ADD Registro Registro Registro
  - ADDI Registro Registro Inmediato
  - LW Registro Relativo

Campo Registro

- Nombre del registro utilizado:
  - Si es directo por registro
  - Si es indirecto por registro
  - Si es relativo
- Tamaño: de 2 a 6 bits
  - 2 bit(procesador 8086, con registros AX, BC, CX, DX)
  - **5 bit** En procesadores con 32 registros, como RiscV

Campo numérico

- En modos relativo → Desplazamiento
- En modos inmediatos → Valor
- En modo directo a memoria →Dirección (descartado)
- Casi siempre un natural o entero
- En RISCV→**Compl-2**

Ejemplo: ADDI en MIPS

La instrucción suma un valor inmediato al registro fuente y guarda el resultado en destino: destino = fuente + inmediato

- ADDI Rg destino, Rg fuente, Inmediato
- MIPS admite 6 bits de opcode
- Operandos 1 y 2 → Modo registro (directo por registro)
  - Hay 32 registros
- Operando 3 → Modo inmediato
  - Entero de 16 bits
- OOOOOO RRRRR RRRRR IIIIIIIIIIIIIIII
- 001000 01001 10111 1111111111111100
  - ADDI R23, R9, #-4

RISC-V

Computador de conjunto reducido de instrucciones, versión 5

- Mantenido por RISC-V International, cualquier empresa puede diseñar, fabricar y vender chips RISC-V sin pagar licencias
  - Procesador de 32 bits (tamaño típico de registros y buses). Aunque hay versiones superiores.
  - Arquitectura Load-Store: Solo las instrucciones de carga y almacenamiento acceden a la memoria; el resto opera sobre registros.
  - Formato de Instrucción Fijo: Facilita el diseño del decodificador en el hardware, lo que reduce el consumo de energía y el área del chip.
  - Sin Modos de Direccionamiento Complejos: Mantiene el ciclo de ejecución predecible y eficiente.
- 32 registros
  - Registros de 32 bits
  - El registro x0 siempre almacena la constante cero
- Reserva 7 bits de opcode: teóricamente, hasta 128 instrucciones, extensibles con funciones

RISC-V REDUCIDO

- Procesador RISC-V (32 bits) con conjunto reducido de instrucciones

	`lw rd, imm`$_{12b}$`(rs1)`	rd ← MEM[rs1 + sExt(imm)]	tipo-I
	`sw rs2, imm`$_{12b}$`(rs1)`	MEM[rs1 + sExt(imm)] ← rs2	tipo-S
ALU	`add rd, rs1, rs2`	rd ← rs1 + rs2	tipo-R
ALU	`sub rd, rs1, rs2`	rd ← rs1 - rs2	tipo-R
ALU	`and rd, rs1, rs2`	rd ← rs1 & rs2	tipo-R
ALU	`or rd, rs1, rs2`	rd ← rs1 | rs2	tipo-R
ALU	`slt rd, rs1, rs2`	rd ← if (rs1 < rs2) then 1 else 0	tipo-R
ALUI	`addi rd, rs1, imm`$_{12b}$	rd ← rs1 + sExt(imm)	tipo-I
ALUI	`andi rd, rs1, imm`$_{12b}$	rd ← rs1 & sExt(imm)	tipo-I
ALUI	`ori rd, rs1, imm`$_{12b}$	rd ← rs1 | sExt(imm)	tipo-I
ALUI	`slti rd, rs1, imm`$_{12b}$	rd ← if (rs1 < sExt(imm)) then 1 else 0	tipo-I
	`beq rs1, rs2, imm`$_{13b}$	PC ← if (rs1 = rs2) then (PC + sExt(imm$_{12:1}$ << 1)) else (PC + 4)	tipo-B
	`jal rd, imm`$_{21b}$	PC ← PC + sExt(imm$_{20:1}$ << 1), rd ← PC + 4	tipo-J

RISC-V REDUCIDO

- Estos son los distintos tipos de formato de instrucciones

31 … 25	24 … 20	19 … 15	14 … 12	11 … 7	6 … 0	
funct7	rs2	rs1	funct3	rd	op	tipo-R
$imm_{11:0}$		rs1	funct3	rd	op	tipo-I
$imm_{11:5}$	rs2	rs1	funct3	$imm_{4:0}$	op	tipo-S
$imm_{12,10:5}$	rs2	rs1	funct3	$imm_{4:1,11}$	op	tipo-B
$imm_{20,10:1,11,19:12}$				rd	op	tipo-J

	Instrucción	Tipo	funct7 (31:25)	funct3 (14:12)	op (6:0)
	lw	I	-	010	0000011
	sw	S	-	010	0100011
ALU	add	R	0000000	000	0110011
ALU	sub	R	0100000	000	0110011
ALU	and	R	0000000	111	0110011
ALU	or	R	0000000	110	0110011
ALU	slt	R	0000000	010	0110011
ALUI	addi	I	-	000	0010011
ALUI	andi	I	-	111	0010011
ALUI	ori	I	-	110	0010011
ALUI	slti	I	-	010	0010011
	beq	B	-	000	1100011
	jal	J	-	-	1101111

En resumen…

- Las instrucciones son secuencias binarias
  - Típicamente, 32 bits (RiscV)
- La secuencia binaria se divide en campos
  - Los tamaños de los campos dependen del hardware
  - Ej.: 5 bits para un registro (RiscV), porque hay 32 registros
- El **formato** de instrucción es
  - Una descripción formal del número, tamaño y posición de los campos específicos de una arquitectura (familia de procesadores compatibles)
  - La relación formato de instrucción/hardware es estrecha:
    - El formato es la abstracción del hardware para el software
    - El formato condiciona el hardware
    - El hardware condiciona el formato

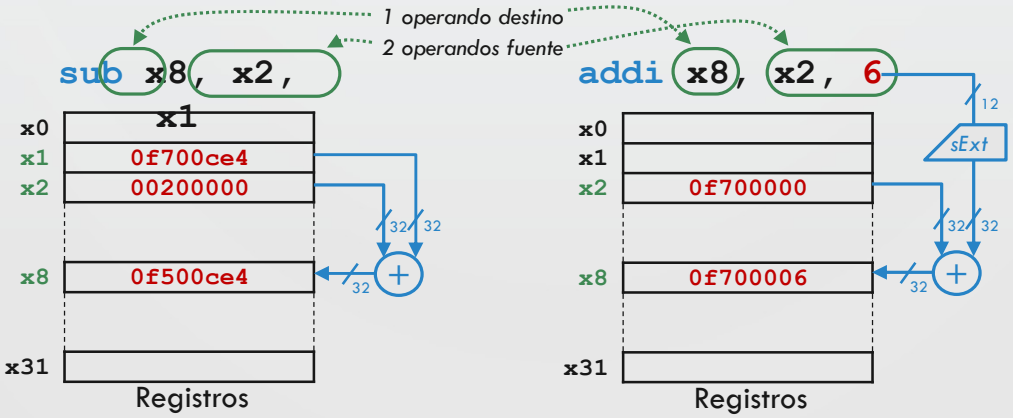
El repertorio de Instrucciones

REPERTORIO DE INSTRUCCIONES

- El repertorio de instrucciones es el conjunto de todas las instrucciones que puede ejecutar un procesador
- Hay diferentes tipos de instrucciones:
  - Transformación de datos: realizan operaciones aritméticas, lógicas bit a bit y de desplazamiento de bits.
  - Transferencia de datos: copian datos entre registros y memoria
  - Control o salto: rompen el orden implícito de ejecución (modifican PC)
  - Privilegiadas: permiten acceso a funcionalidades para control del sistema
- El repertorio de instrucciones del RISC-V es extremadamente reducido

REPERTORIO DE INSTRUCCIONES:
ARITMÉTICAS (I)

- Realizan operaciones aritméticas con 2 operandos fuente y 1 operando destino, todos de 32 bits
  - El operando izquierdo se encuentra SIEMPRE en un registro
  - El operando derecho se encuentra en un registro o es un inmediato corto
    - La constante inmediata es de 12 bits en C2 ([-2048, +2047]) pero se extiende a 32 bits
  - El resultado se almacena SIEMPRE en un registro



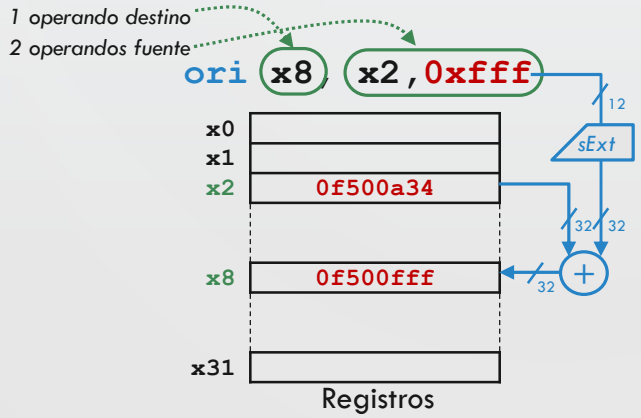
REPERTORIO DE INSTRUCCIONES:
ARITMÉTICAS (II)

Instrucción	Operación	Descripción
`add rd, rs1, rs2`	rd ← rs1 + rs2	add suma
`sub rd, rs1, rs2`	rd ← rs1 - rs2	subtract resta
`slt rd, rs1, rs2`	rd ← if (rs $<_S$ rs2) then (1) else (0)	set if less than "menor que" con signo
`sltu rd, rs1, rs2`	rd ← if (rs $<_U$ rs2) then (1) else (0)	set if less than unsigned "menor que" sin signo
`addi rd, rs1, imm`$_{12b}$	rd ← rs1 + sExt(imm)	add immediate Suma una constante
`slti rd, rs1, imm`$_{12b}$	rd ← if (rs $<_S$ sExt(imm)) then (1) else (0)	set if less than immediate "menor que constante" con signo
`sltiu rd, rs1, imm`$_{12b}$	rd ← if (rs $<_U$ sExt(imm)) then (1) else (0)	set if less than immediate unsigned "menor que constante" sin signo

- Existen instrucciones diferentes de comparación para interpretar operandos fuente como datos con y sin signo

REPERTORIO DE INSTRUCCIONES:
LÓGICAS (I)

- Realizan operaciones de tipo lógico bit a bit (bitwise) con 2 operandos fuente y 1 operando destino, todos de 32 bits
  - El operando izquierdo se encuentra SIEMPRE en un registro
  - El operando derecho se encuentra en un registro o es un inmediato corto
    - La constante inmediata es de 12 bits en C2 pero se extiende su signo a 32 bits
  - El resultado se almacena SIEMPRE en un registro




```
   00001111010100000000101000110100
or 00000000000000000000111111111111
   00001111010100000000111111111111
```

REPERTORIO DE INSTRUCCIONES: LÓGICAS (II)

- Las operaciones de tipo lógico bit a bit se usan para manipular los bits individuales de un dato
  - Un operando contiene el dato a manipular
  - El otro operando contiene una máscara que indica los bits a cambiar
  - Se utilizará una operación distinta según el cambio que se desee

La instrucción **or** *pone a 1 aquellos bits del dato cuyos correspondientes en la máscara estén a 1*

```
    00001111010100000000101000110100   <------- dato
or  00000000000000000000111111111111   <------- máscara
    00001111010100000000111111111111   <------- dato manipulado
```

La instrucción **and** *pone a 0 aquellos bits del dato cuyos correspondientes en la máscara estén a 0*

```
    00001111010100000000101000110100
and 00000000000000000000111111111111
    00000000000000000000101000110100
```

La instrucción **xor** *complementa aquellos bits del dato cuyos correspondientes en la máscara estén a 1*

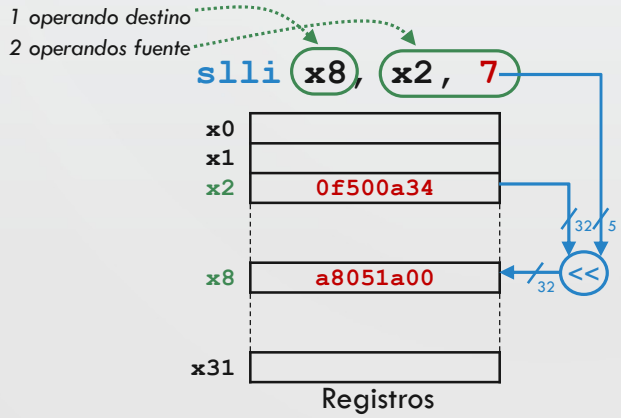
```
    00001111010100000000101000110100
xor 00000000000000000000111111111111
    00001111010100000000010111001011
```

REPERTORIO DE INSTRUCCIONES:
LÓGICAS (III)

Instrucción	Operación	Descripción
`and rd, rs1, rs2`	rd ← rs1 & rs2	and "y lógica" bit a bit
`or rd, rs1, rs2`	rd ← rs1 | rs2	or "o lógica" bit a bit
`xor rd, rs1, rs2`	rd ← rs1 ^ rs2	xor "o-exclusiva lógica" bit a bit
`andi rd, rs1,` imm_{12b}	rd ← rs1 & sExt(imm)	and immediate "y lógica" bit a bit con constante
`ori rd, rs1,` imm_{12b}	rd ← rs1 | sExt(imm)	or immediate "o lógica" bit a bit con constante
`xori rd, rs1,` imm_{12b}	rd ← rs1 ^ sExt(imm)	xor immediate "o-exclusiva lógica" bit a bit con constante

REPERTORIO DE INSTRUCCIONES:
DE DESPLAZAMIENTO (I)

- Permiten desplazar los bits de un operando fuente un número de posiciones indicadas por otro y almacenarlo en un operando destino
  - El operando izquierdo de 32 bits se encuentra SIEMPRE en un registro
  - El operando derecho de 5 bits se encuentra en un registro o es un inmediato
    - Del registro se toman los 5 bits menos significativos
    - La constante inmediata son 5 bits en binario puro que no se extienden
  - El resultado se almacena SIEMPRE en un registro



REPERTORIO DE INSTRUCCIONES:
DE DESPLAZAMIENTO (II)

- Insertan 0's por un extremo del dato y descartan los bits que salen por el otro
  - Permiten el reescalado de datos sin signo:
    - Desplazar n bits a la izquierda equivale a multiplicar por 2^n
    - Desplazar n bits a la derecha equivale a dividir por 2^n

Desplazar cada bit 7 posiciones a la izquierda

```
     00000000000000000000101000110100
slli                            00111
     00000000000001010001101000000000
```

$2612 << 7 = 2612 \times 2^7 = 334336$

Desplazar cada bit 7 posiciones a la derecha

```
     00000000000000000000101000110100
srli                            00111
     00000000000000000000000000010100
```

$2612 >> 7 = 2612 \div 2^7 = 20$

- Aplicadas tras una lógica bit a bit permiten extraer campos de un dato:

```
     00001111101000000001010001101000
and  00000000111100000000000000000000
     00000000101000000000000000000000
srli                            10100
     00000000000000000000000000001010
```

Aísla los bits 20 al 23 del dato

Los desplaza al extremo derecho (bits menos significativos)

REPERTORIO DE INSTRUCCIONES:
DE DESPLAZAMIENTO (III)

- La instrucción de desplazamiento aritmético a la derecha propaga el bit de signo por la izquierda y descarta los bits que salen por la derecha
  - Permite el reescalado de datos con signo:

Desplazar cada bit 7 posiciones a la derecha

```
     11111111111111110001101000110100
srai                            00111
     11111111111111111111111000110100
```

$-58828 >> 7 = -58828 \div 2^7 = -460$

Desplazar cada bit 7 posiciones a la derecha

```
     01111111111111110001101000110100
srai                            00111
     00000000111111111111111000110100
```

$2147424820 >> 7 = 2147424820 \div 2^7 = 16776756$

- No existe instrucción de desplazamiento aritmético a la izquierda
  - Para resultados válidos (el dato reescalado con signo puede representarse con 32 bits) el desplazamiento lógico es equivalente

Desplazar cada bit 7 posiciones a la izquierda

```
     11111111111111110001101000110100
slli                            00111
     11111111100011010001101000000000
```

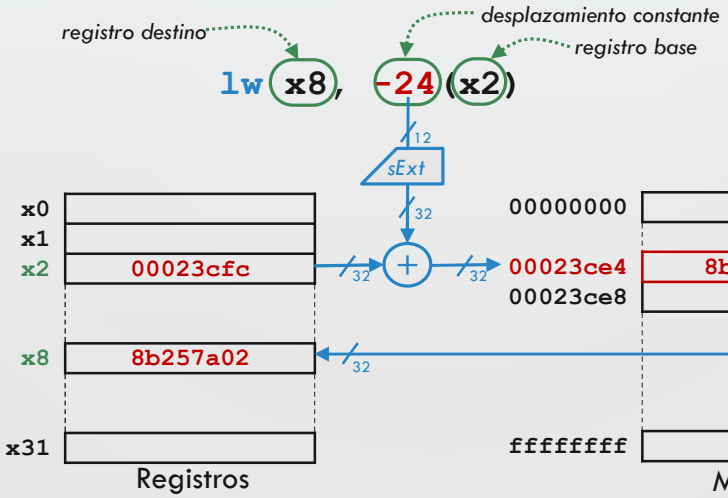
$-58828 << 7 = -58828 \times 2^7 = -7529984$

REPERTORIO DE INSTRUCCIONES:
DE DESPLAZAMIENTO (IV)

Instrucción	Operación	Descripción
`sll rd, rs1, rs2`	rd ← rs1 << $rs2_{4:0}$	shift left logical desplazamiento lógico a izquierda
`srl rd, rs1, rs2`	rd ← rs1 >> $rs2_{4:0}$	shift right logical desplazamiento lógico a derecha
`sra rd, rs1, rs2`	rd ← rs1 >>> $rs2_{4:0}$	shift right arithmetical desplazamiento aritmético a derecha
`slli rd, rs1,` imm_{5b}	rd ← rs1 << imm	shift left logical immediate desplazamiento lógico a izquierda con constante
`srli rd, rs1,` imm_{5b}	rd ← rs1 >> imm	shift right logical immediate desplazamiento lógico a derecha con constante
`srai rd, rs1,` imm_{5b}	rd ← rs1 >>> imm	shift right arithmetical immediate desplazamiento aritmético a derecha con constante

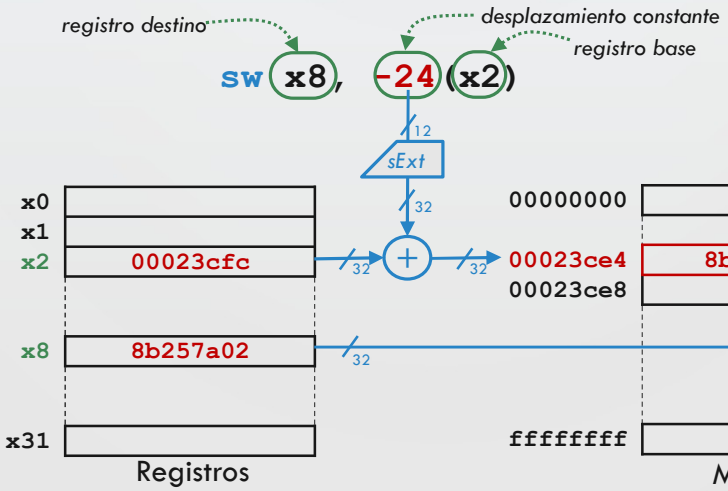
REPERTORIO DE INSTRUCCIONES:
DE TRANSFERENCIA DE DATOS: CARGA

- Permiten copiar un dato de memoria a un registro
  - Utiliza direccionamiento relativo a registro base para indicar la dirección de memoria que ocupa el dato (la dirección es la suma de una dirección base y un desplazamiento)
    - La dirección base se encuentra en un registro
    - El desplazamiento es un inmediato de 12 bits en C2 cuyo signo se extiende a 32 bits
  - El dato leído de memoria se carga en un registro



REPERTORIO DE INSTRUCCIONES:
DE TRANSFERENCIA DE DATOS: ALMACENAMIENTO

- Permiten copiar un dato de un registro a memoria
  - El dato se encuentra en un registro
  - Utiliza direccionamiento relativo a registro base para indicar la dirección de memoria en donde almacenar el dato (la dirección es la suma de una dir. base y un desplazamiento)
    - La dirección base se encuentra en un registro
    - El desplazamiento es un inmediato de 12 bits en C2 cuyo signo se extiende a 32 bits



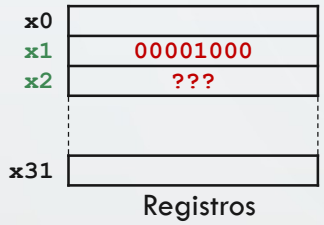
REPERTORIO DE INSTRUCCIONES:
DE TRANSFERENCIA DE DATOS (I)

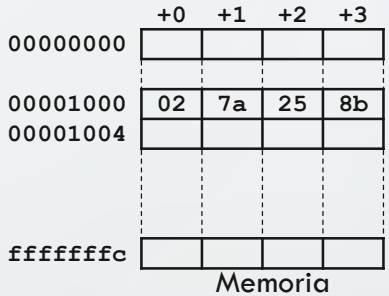
Instrucción	Operación	Descripción
`lw rd, imm`$_{12b}$`(rs1)`	rd ← Mem[rs1 + sExt(imm)]	load word carga palabra
`lh rd, imm`$_{12b}$`(rs1)`	rd ← sExt(Mem[rs1 + sExt(imm)]$_{15:0}$)	load half carga media palabra con signo
`lhu rd, imm`$_{12b}$`(rs1)`	rd ← zExt(Mem[rs1 + sExt(imm)]$_{15:0}$)	load half unsigned carga media palabra sin signo
`lb rd, imm`$_{12b}$`(rs1)`	rd ← sExt(Mem[rs1 + sExt(imm)]$_{7:0}$)	load byte carga byte con signo
`lbu rd, imm`$_{12b}$`(rs1)`	rd ← zExt(Mem[rs1 + sExt(imm)]$_{7:0}$)	load half unsigned carga byte sin signo
`sw rs2, imm`$_{12b}$`(rs1)`	Mem[rs1 + sExt(imm)] ← rs2	store word almacena palabra
`sh rs2, imm`$_{12b}$`(rs1)`	Mem[rs1 + sExt(imm)]$_{15:0}$ ← rs2$_{15:0}$	store whalf almacena media palabra
`sb rs2, imm`$_{12b}$`(rs1)`	Mem[rs1 + sExt(imm)]$_{7:0}$ ← rs2$_{7:0}$	store byte almacena byte

- Existen instrucciones diferentes para copiar datos de 8, 16 o 32 bits
- Existen instrucciones diferentes para cargar datos extendiendo el signo o completando con ceros

REPERTORIO DE INSTRUCCIONES:
DE TRANSFERENCIA DE DATOS (II)

- Los datos en memoria están alineados y con orden Little-Endian

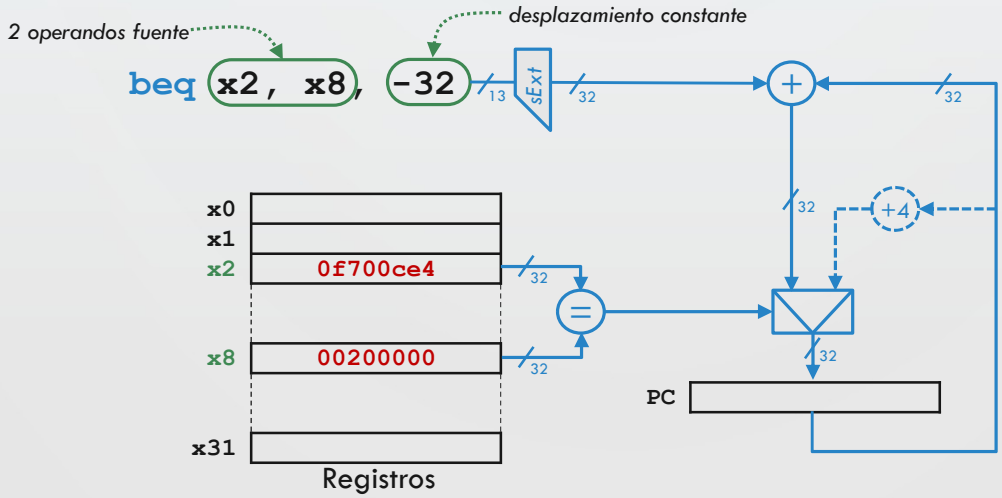




Instrucción	Operación	Descripción
`lw x2, 0(x1)`	x2 ← Mem[x1 + sExt(0)]	carga **8b257a02** en x2
`lhu x2, 0(x1)`	x2 ← zExt(Mem[x1 + sExt(0)]$_{15:0}$)	carga **00007a02** en x2 (7a02 $=_2$ +31234)
`lhu x2, 2(x1)`	x2 ← zExt(Mem[x1 + sExt(2)]$_{15:0}$)	carga 00008**b25** en x2 (8b25 $=_2$ +35621)
`lh x2, 0(x1)`	x2 ← sExt(Mem[x1 + sExt(0)]$_{15:0}$)	carga **00007a02** en x2 (7a02 $=_{C2}$ +31234)
`lh x2, 2(x1)`	x2 ← sExt(Mem[x1 + sExt(2)]$_{15:0}$)	carga **ffff8b25** en x2 (8b25 $=_{C2}$ -29915)
`lbu x2, 3(x1)`	x2 ← zExt(Mem[x1 + sExt(3)]$_{7:0}$)	carga **0000008b** en x2 (8b $=_2$ +139)
`lb x2, 3(x1)`	x2 ← sExt(Mem[x1 + sExt(3)]$_{7:0}$)	carga **ffffff8b** en x2 (8b $=_{C2}$ -177)
`lh x2, 3(x1)`	x2 ← sExt(Mem[x1 + sExt(3)]$_{15:0}$)	*Al ejecutarse provoca error de alineamiento*

REPERTORIO DE INSTRUCCIONES:
DE SALTO CONDICIONAL (I)

- Permiten romper la secuencia normal de ejecución saltando a una dirección cercana cuando se cumple cierta condición
  - Compara dos operandos fuente que se encuentran en registros
  - Utiliza direccionamiento relativo al PC para indicar la dirección con que actualizar el PC en caso de ser cierta la condición
    - Dicha dirección es la suma del contenido del PC y un desplazamiento corto
    - El desplazamiento es una constante de 13 bits en C2 cuyo signo se extiende a 32 bits



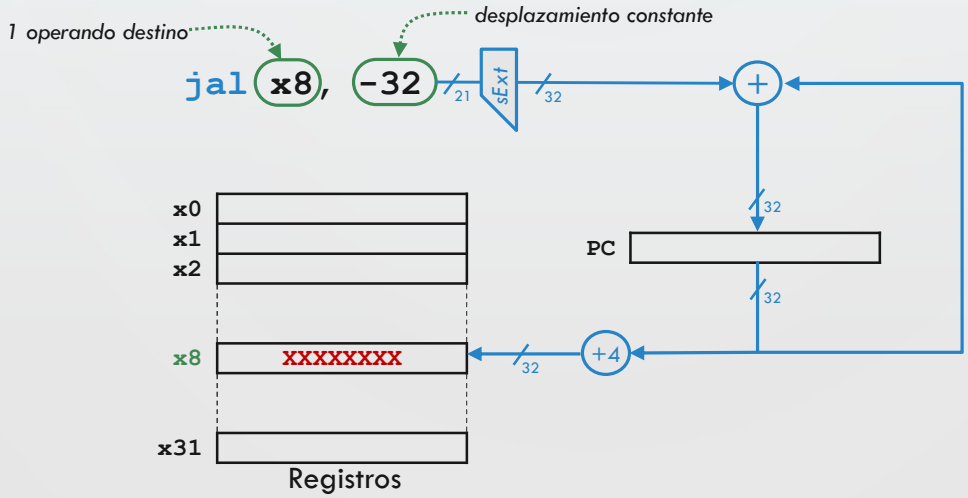
REPERTORIO DE INSTRUCCIONES:
DE SALTO CONDICIONAL (II)

Instrucción	Operación	Descripción
`beq rs1, rs2, imm`$_{13b}$	*if* (rs1 = rs2) *then* (PC ← PC + sExt(imm$_{12:1}$ << 1)	branch if equal salta si "igual que"
`bne rs1, rs2, imm`$_{13b}$	*if* (rs1 ≠ rs2) *then* (PC ← PC + sExt(imm$_{12:1}$ << 1)	branch if not equal salta si "disntinto que"
`blt rs1, rs2, imm`$_{13b}$	*if* (rs1 $<_S$ rs2) *then* (PC ← PC + sExt(imm$_{12:1}$ << 1)	branch if less than salta si "menor que" con signo
`bge rs1, rs2, imm`$_{13b}$	*if* (rs1 $\geq_S$ rs2) *then* (PC ← PC + sExt(imm$_{12:1}$ << 1)	branch if greater than or equal salta si "mayor o igual que" con signo
`bltu rs1, rs2, imm`$_{13b}$	*if* (rs1 $<_U$ rs2) *then* (PC ← PC + sExt(imm$_{12:1}$ << 1)	branch if less than unsigned salta si "menor que" sin signo
`bgeu rs1, rs2, imm`$_{13b}$	*if* (rs1 $\geq_U$ rs2) *then* (PC ← PC + sExt(imm$_{12:1}$ << 1)	branch if greater than or equal unsigned salta si "mayor o igual que" sin signo

- Las instrucciones de salto condicional se usan para implementar en ensamblador estructuras de control tipo *if*, *while*, *for*, …
- Existen instrucciones diferentes para interpretar los operandos fuente que se comparan como datos con y sin signo

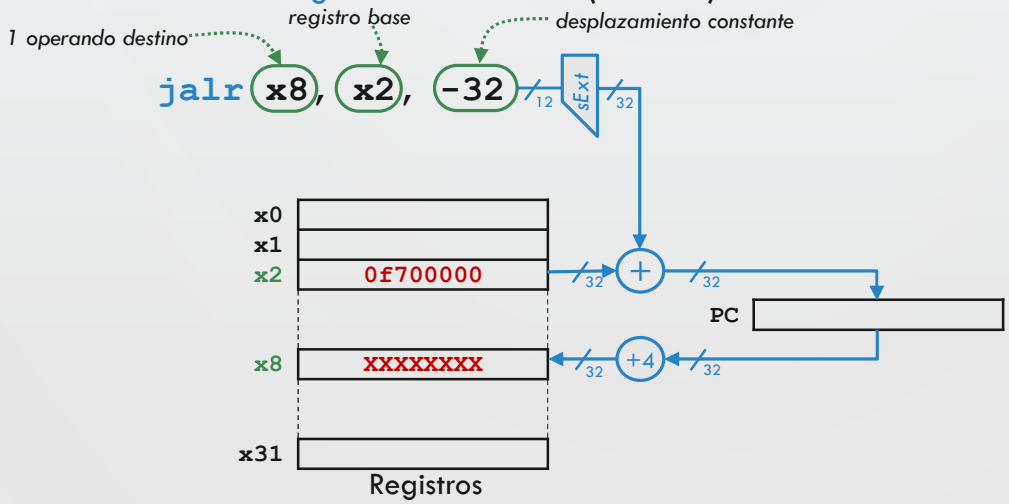
REPERTORIO DE INSTRUCCIONES:
DE SALTO A FUNCIÓN: JAL

- Permiten romper la secuencia normal de ejecución saltando a una dirección, pero guardando la dirección de retorno
  - Utiliza direccionamiento relativo al PC para indicar la dirección con que actualizar el PC
    - Dicha dirección es la suma del contenido del PC y un desplazamiento largo
    - El desplazamiento es una constante de 21 bits en C2 cuyo signo se extiende a 32 bits
  - La dirección de la siguiente instrucción (retorno) se almacena en un registro



REPERTORIO DE INSTRUCCIONES:
DE SALTO A FUNCIÓN: JALR

- Permiten romper la secuencia normal de ejecución saltando a una dirección, pero guardando la dirección de retorno
  - Utiliza direccionamiento relativo a base para indicar la dirección con que actualizar el PC
    - Dicha dirección es la suma del contenido del registro y un desplazamiento corto
    - El desplazamiento es una constante de 12 bits en C2 cuyo signo se extiende a 32 bits
  - La dirección de la siguiente instrucción (retorno) se almacena en un registro



REPERTORIO DE INSTRUCCIONES:
DE SALTO A FUNCIÓN (II)

Instrucción	Operación	Descripción
`jal rd, imm`$_{21b}$	PC ← PC + sExt(imm$_{20:1}$ << 1), rd ← PC+4	jump and link salto a función con dirección relativa a PC
`jalr rd, rs1, imm`$_{12b}$	PC ← rs1 + sExt(imm), rd ← PC+4	jump and link register salto a función con dirección relativa a registro base

- En el repertorio del RISC-V no existen instrucciones de retorno desde función, ni de salto incondicional a una instrucción, pero pueden usarse éstas para hacerlo:
  - Suponiendo que la instrucción de retorno está almacenada en el registro **xn**, el retorno se hace con:

    `jalr x0, xn, 0`

  - El salto incondicional (relativo a PC) a la dirección que ocupa cierta instrucción se hace con:

    `jal x0, imm`$_{21b}$